Build prompt — thinking-first local AI for coding

What this is. The master instruction for AI coding agents (VS Code: Copilot agent mode, Cline, Codex, Claude Code or similar) that will build the system. Section 1 is the document set — every file the agents must have on disk. Section 2 is the standing prompt that applies to every session. Section 3 is the build, phase by phase, each with its reading list, tasks, acceptance test and a copy-paste prompt to open the session. Section 4 is what the human does between phases.

Owner: Matěj Forejt. **Repository:** `C3Studium/tfa` (working name “tfa” — thinking-first assistant; rename freely before the first commit). **Machine:** Windows 11, Ryzen 9 7900X, RTX 3080 12 GB, 32 GB DDR5, 1 TB NVMe. **Language:** TypeScript on Node ≥ 20 for everything except training, which is Python.

1. The document set

Put this entire folder at `docs/` in the repository. Agents read from there; nothing is read from the web unless a document says “verify in paper”.

#	File	What it is	Who reads it
0	`docs/thinking-first-local-ai-thesis.pdf`	The design: why, architecture, phases, acceptance tests, risks	Everyone, first
1	`docs/pdf/00-index.pdf`	Map from architecture sections to packs; reading order; conventions	Everyone
2	`docs/pdf/10-interface-contract.pdf`	**Normative.** Repository layout, store formats, HTTP contract, module interfaces, event flow, invariants I-1…I-10, config	Everyone, before writing code
3	`docs/pdf/20-agent-build-prompt.pdf`	This document	Everyone
4	`docs/pdf/A-context-memory.pdf`	Notebook, reflector/curator, core block, sleep — ACE, Reflexion, Dynamic Cheatsheet, A-MEM, AWM, Generative Agents, MemGPT, sleep-time compute	P3, P4
5	`docs/pdf/B-skills-program-synthesis.pdf`	Skill library — Voyager, SoK agentic skills, DreamCoder, CodeAct, LLMs as tool makers	P3

#	File	What it is	Who reads it
6	`docs/pdf/C-verified-search.pdf`	Thinking tier — LATS, Tree of Thoughts, self-consistency, CodeT, AlphaCode, Large Language Monkeys, compute-optimal test-time scaling	P2
7	`docs/pdf/D-self-training.pdf`	Weekly LoRA — STaR, ReST-EM, DPO, Absolute Zero, LoRA, QLoRA, distillation, self-rewarding	P5
8	`docs/pdf/E-test-time-learning.pdf`	Learning while working — SEAL, TTT on ARC, Titans, TTT layers, Transformer², Nested Learning/HOPE	P6
9	`docs/pdf/F-recursive-self-improving.pdf`	HRM, TRM, CTM, Darwin Gödel Machine, Adaptive Computation Time; the integrity lesson	P5 (harness evolution), P6
10	`docs/pdf/G-serving-efficiency-grounding.pdf`	Serving: speculative decoding, FlashAttention, MoE, GPTQ/AWQ, KV cache; Library: RAG, ColBERT, lost-in-the-middle; memory/bandwidth arithmetic	P0, P1, P4
11	`docs/pdf/H-routing-evaluation.pdf`	Gate and benchmark — RouteLLM, FrugalGPT, SWE-bench/Verified, measure of intelligence, SWE-agent	P2
12	`docs/pdf/I-foundations-long-term.pdf`	Thousand Brains, I-JEPA, V-JEPA 2, Alberta Plan, Lake et al., BabyLM, Standard Model of the Mind	P6; background
13	`docs/markdown/*.md`, `docs/markdown/*.json`	The same packs and contract as Markdown (LaTeX math) and JSON source tables — **the machine-readable copies agents should load into context**	Everyone
14	`docs/sources/sources.csv`	61 papers: section, arXiv id, title, PDF URL, code URL	Everyone
15	`docs/sources/download-papers.ps1` (`.sh`)	Fetches every original PDF into `docs/papers/<section>/`	Human, once
16	`docs/papers/<section>/*.pdf`	The original papers (after running 15)	Agents, when a pack says “verify in paper”

#	File	What it is	Who reads it
17	`docs/sources/BRIEF.md`, `docs/sources/manifest.json`	How the packs were extracted; the section → paper map	Reference

Reading order for any agent joining the project: 0 → 1 → 2 → 3 → the pack(s) for its phase → the papers only where a pack flags a detail to verify.

---

2. Standing prompt (applies to every session, every phase)

Paste this at the top of every agent session, followed by the phase prompt from Section 3.

```
You are building "tfa", a local, verified, self-improving coding assistant, in the repository you
are in.
Read, in this order, before any code: docs/pdf/00-index.pdf (or docs/markdown/00-index.md),
docs/pdf/10-interface-contract.pdf (normative), docs/pdf/20-agent-build-prompt.pdf, and the
knowledge
pack(s) named in the phase prompt. The thesis docs/thinking-first-local-ai-thesis.pdf is the
design rationale.

RULES
1. The interface contract is normative. Implement its types, file formats, HTTP contract, event
flow and
   invariants I-1..I-10 exactly. If you need to deviate, stop and write the reason to ASK-
HUMAN.md.
2. Every algorithm or constant taken from a paper carries a comment  // arXiv:<id> §<section>
(I-10).
   The packs are the source; do not reinvent what a pack already specifies.
3. Where a pack says "(not in fetched text — verify in paper §n)" and you need that detail, open
the
   original PDF under docs/papers/ and verify it. If the PDF is not there, write to ASK-HUMAN.md
and
   use the pack's stated default, marked  // UNVERIFIED  in code.
4. Human-owned paths (src/verify/**, src/bench/**, .tfa/bench/**, lesson tests) are read-only to
you.
   Changes to them go into proposals/<id>.patch with a one-paragraph justification. Never weaken
a test,
   a verifier stage or a benchmark task to make something pass.
5. Every feature ships with vitest tests that run without a GPU: fixture repositories plus the
mock
   model server in test/helpers.ts. The real models are exercised only in the phase acceptance
test,
   run by the human.
6. Keep dependencies minimal: typescript, tsx, vitest, @types/node; better-sqlite3 for the
Library;
   nothing else without writing the reason to ASK-HUMAN.md. Training scripts are Python with
   transformers, peft, trl, bitsandbytes (or unsloth); nothing else.
7. Windows is the target. Spawn commands through the shell; no file symlinks (directory junctions
are
   fine); paths via node:path; CRLF-safe parsing.
8. Log everything the contract says to log. A model call without a trajectory event is a bug.
9. Small, verified steps: typecheck + tests green after every task; commit per task with the
message
   format  "<phase>/<task>: <what> (<arXiv ids used>)".
10. When unsure, ask: append to ASK-HUMAN.md (question, options, your recommendation) and
continue with
    the next independent task. Never guess on anything that touches the verifier, the benchmark,
data
    deletion, or spending GPU time.

DEFINITION OF DONE for a task: contract types implemented; tests green; trajectory events written
where
```

```
applicable; README section updated; the acceptance test of the phase runnable by `npm run`
scripts.
```

---

3. The build, phase by phase

Each phase ends with the acceptance test from the thesis (Table 5). A later phase never starts before the earlier one's test passes on the real machine, run by the human. Durations are rough and part-time.

P0 — Reader + harness (exists; adopt and harden) · ~1 week

Read: contract §4, §5, §9; Pack G §2 (serving layout, prefix caching, speculative decoding); Pack C §1.4 (CodeT, for the edit/verify pattern).

State: a P0 scaffold exists (`tfa-p0.zip`: config, llama.cpp client, JSON edit format with rollback, cheapest-first cascade, type-check-driven context, fix loop with reflection, trajectory log, CLI, 15 tests). Either adopt it as the first commit or rebuild to the contract; if adopting, migrate its trajectory events to contract §5 and its config to §10.

Tasks 1. Repository bootstrap under `C3Studium/tfa`: layout per contract §2; CI running typecheck + vitest on Windows and Linux runners. 2. `src/llm/client.ts` to contract §4 (slots, `cache_prompt`, `id_slot`, `/health`, embeddings, rerank); `Reader` interface. 3. `src/verify/*` to contract §7 `Verifier` — human-owned from the first commit (I-1 write guard + test). 4. `src/log.ts` to contract §5 (all event types; append-only; log tails). 5. `tfa doctor | verify | fix | models`. 6. `tfa models` prints the download and `llama-server` commands for worker (Qwen3.5-9B Q6_K), draft (Qwen3.5 ≤2B), senior A (GLM-4.7-Flash) and B (Qwen3.6-35B-A3B) with expert offload, embed, rerank; file names to be confirmed on Hugging Face by the human.

Acceptance (human runs): `tfa fix <repo> "<task>"` repairs a real TypeScript error in one of the owner's repositories and `npm run build` passes. Record attempts and wall-clock in `bench/runs/p0-first.json`.

Session prompt

```
PHASE P0. Read docs/markdown/10-interface-contract.md fully and docs/markdown/G-serving-
efficiency-grounding.md §2.
Adopt the P0 scaffold in this repository (or rebuild it) so that it matches the contract: §4
model server contract,
§5 trajectory events, §7 Reader/Verifier interfaces, §9 invariants I-1, I-4, I-6, I-8, I-9, §10
config.
Deliver the task list P0.1–P0.6 from docs/markdown/20-agent-build-prompt.md with tests. Do not
implement anything
from later phases. Finish by updating README.md with the P0 acceptance test instructions for the
human.
```

P1 — Library (grounding) · ~1–2 weeks

Read: contract §6.3, §7 `Retriever`; Pack G §1.7–1.9 (lost-in-the-middle, RAG, ColBERT) and §2 (retrieval pipeline, memory arithmetic).

Tasks 1. `retrieve/chunk.ts`: code-aware chunking (function/component/export via the TypeScript compiler API) and heading-based chunking for Markdown/MDX docs; chunk hashes. 2. `retrieve/index.ts`: SQLite (better-sqlite3) with FTS5 for lexical search, a vector table for embeddings (brute-force cosine is fine at < 200k chunks), and the symbol table; incremental reindex by content hash; `library/manifest.json`. 3. `retrieve/versions.ts`: read `package.json` ranges; tag doc chunks with the package and version they document; filter before scoring. 4. `retrieve/hybrid.ts`: BM25 ∪ dense → reciprocal-rank fusion → `rerank` top-40 → pack under budget; citations recorded in `Ev.retrieve`. 5. Ingestion CLI: `tfa index <repo|docs-folder> [--pkg next@15.x]`; a starter ingest of

the owner’s repos plus the Next.js, React and TypeScript docs for the versions in use. 6. Replace P0’s type-check-only context with `Retriever` output in the fixed prompt order (I-9), under `budgets.contextTokens`.

Acceptance (human runs): ten questions about the owner’s own CMS answered with the right file cited and zero invented APIs (checked by `tsc` on any code in the answers).

Session prompt

```
PHASE P1. Read docs/markdown/10-interface-contract.md §6.3, §7 (Retriever) and docs/markdown/G-
serving-efficiency-grounding.md
§1.7–1.9 and §2. Implement tasks P1.1–P1.6 from docs/markdown/20-agent-build-prompt.md. Version-
aware filtering
precedes scoring; retrieval hits are logged with cited:true/false; context is packed under
budgets.contextTokens
in the fixed prompt order. Tests: fixture docs with two versions of the same API; assert the
wrong version is never
retrieved; assert symbol lookup is exact. Do not touch src/verify or src/bench.
```

P2 — Loop + verifier + benchmark (the baseline) · ~2 weeks

Read: contract §6.4, §6.5, §7 (`Decomposer`, `Gate`, `Thinker`, `Bench`), §8; Pack C §1.1 (LATS), §1.4 (CodeT), §1.6–1.7 (sampling laws, compute-optimal), §2; Pack H §1.1–1.3, §1.5, §2 (gate features, benchmark protocol).

Tasks 1. `Decomposer`: task → subtasks with `touches[]` and `size`; senior re-decomposition above threshold. 2. `Gate` in rule mode (contract §6.4 cold-start rule); `gate/decisions.jsonl` with outcomes; asymmetric escalation (fast → slow after two verification failures). 3. `Thinker.search`: plan-level sampling (N plans), implement each, score with the cascade cheapest-first, expand survivors (UCT with the LATS constants, cited), anytime stop on agreement; budgets by difficulty from config; senior as critic before author. 4. Prose/citation stage: `citation` stage checks that every claim in a prose answer cites a retrieved chunk id; unverifiable outputs are returned with `unverified:true`. 5. `tfa task <repo> "<text>"` implementing the full event flow of contract §8 (without reflect/save, which are P3 — write the events as no-ops). 6. `Bench` (human-owned): `bench/tasks` format, runner that checks out `baseCommit` in a worktree, runs the system, grades FAIL_TO_PASS / PASS_TO_PASS (Pack H: SWE-bench), writes `bench/runs`; `tfa bench dev|holdout`. 7. Cost accounting: wall-clock, tokens, senior calls, stages; watt-hours if a meter is available (optional).

Acceptance (human runs): 20 dev + 10 holdout tasks exist (human-written); `tfa bench dev` produces the baseline run with first-try pass rate, corrections, fast-path share and cost per task.

Session prompt

```
PHASE P2. Read docs/markdown/10-interface-contract.md §6.4, §6.5, §7, §8; docs/markdown/C-
verified-search.md §1.1, §1.4,
§1.6, §1.7, §2; docs/markdown/H-routing-evaluation.md §1.1–1.3, §1.5, §2. Implement tasks P2.1–
P2.7 from
docs/markdown/20-agent-build-prompt.md. The bench runner (P2.6) is human-owned: write it under
src/bench and then
treat it as read-only. Constants from LATS/CodeT/RouteLLM carry arXiv comments. Tests: mock model
server returning
scripted plans and edits; fixture repo with tests; assert cheapest-first ordering, survivor
expansion, anytime stop,
and that a skipped stage never counts as a pass (I-4).
```

P3 — Notebook + skills (the system becomes yours) · ~3 weeks

Read: contract §6.1, §6.2, §7 (`Notebook`, `Reflector`, `Curator`, `SkillStore`), §8 steps 3.6–3.8, §9 I-3, I-5; Pack A §1.1 (ACE), §1.2 (Reflexion), §1.4 (A-MEM), §1.6 (Generative Agents scoring), §2; Pack B §1.1 (Voyager), §1.2 (SoK), §2.

Tasks 1. `memory/notebook.ts`: append-only ops, rebuilt index, retrieval scoring (recency 0.995^h, importance, relevance; Pack A), `uses`/`lastUsed` updates; `core.md` rendering stub (sleep fills it in P4). 2. `memory/reflector.ts`: from a trajectory + result, produce lesson and playbook deltas in ACE's bullet format with helpful/harmful counters; the prompt lives in `src/llm/prompts/`. 3. `memory/curator.ts`: deterministic merge of deltas into ops (`add`, `count`, `link` only); embedding dedupe at the Pack A threshold. 4. Lessons-as-tests: for a lesson with a compilable check, generate the assertion/test file and write it as a **proposal** to the human-owned lesson-tests directory (I-1). 5. `skills/*`: contract §6.2 — match, apply, save rule (clean-worktree re-run, parameterization + test, dedupe), use counts; description embeddings cached by hash. 6. Corrections pipeline: detect human edits to touched files within 24 h → `Ev.correction` → lesson delta + test proposal + preference pair (contract §6.7). 7. Wire reflect/save into `tfa task` (contract §8 steps 3.6–3.8) and the fast path (skill apply) into the gate.

Acceptance (human runs): the 20 dev tasks re-run: first-try pass rate up, corrections down, fast-path share measurable and > 0. If not, stop and fix the memory design before P4.

Session prompt

```
PHASE P3. Read docs/markdown/10-interface-contract.md §6.1, §6.2, §6.7, §7, §8, §9;
docs/markdown/A-context-memory.md
§1.1, §1.2, §1.4, §1.6, §2; docs/markdown/B-skills-program-synthesis.md §1.1, §1.2, §2. Implement
tasks P3.1–P3.7 from
docs/markdown/20-agent-build-prompt.md. Notebook and skill stores are append-only operation logs
(I-5); nothing is
saved without a VerifyResult (I-3); lesson tests are proposals, never direct writes (I-1). Tests:
replay a recorded
trajectory through reflector → curator and snapshot the ops; skill save must fail when the clean-
worktree re-run fails;
correction detection on a fixture git history.
```

P4 — Sleep + serving efficiency · ~3 weeks

Read: contract §6.8, §7 (`Sleeper`), §4 (senior lifecycle, slot save); Pack A §1.8 (sleep-time compute), §1.7 (MemGPT paging), §2.4; Pack G §1.1 (speculative decoding), §2 (prefix caching, KV quantization, offload arithmetic).

Tasks 1. `sleep/consolidate.ts`: replay unreflected trajectories; merge near-duplicate notes/skills (embedding + senior judgment); decay by half-life; promote by utility into `core.md` under `coreTokens`; flag contradictions (never auto-resolve); incremental reindex. 2. `sleep/prethink.ts`: for queued tasks, run retrieval and plan drafts and cache them in the trajectory. 3. `sleep/report.ts`: the morning note. 4. Idle detection and budgeting; interruptible; presence throttling. 5. Serving: `llama-server` launch profiles with prefix slot save/restore, KV q8, draft model for the worker; senior on-demand load/unload; A vs B senior benchmarked on the dev set and the winner written to config. 6. Queue: `tfa queue add|run` for overnight tasks with the `queue` budget.

Acceptance (human runs): after a month of use, `notebook/index.json` is smaller than the raw trajectory log, the dev-set curve is still rising, and the fast path handles the majority of routine tasks.

Session prompt

```
PHASE P4. Read docs/markdown/10-interface-contract.md §4, §6.8, §7 (Sleeper); docs/markdown/A-
context-memory.md §1.7, §1.8,
§2.4; docs/markdown/G-serving-efficiency-grounding.md §1.1, §2. Implement tasks P4.1–P4.6 from
docs/markdown/20-agent-build-prompt.md. Sleep-only ops (merge, retire, promote) are the only
writers of core.md;
contradictions are flagged to the human, never resolved. Tests: consolidation on a fixture
notebook with duplicates
and stale notes; core block stays under budget; sleep stops within its ms budget when
interrupted.
```

P5 — Self-training + harness evolution · ~4 weeks

Read: contract §6.6, §6.7, §7 (`Trainer`), §9 I-2, I-7; Pack D §1.1 (STaR), §1.2 (ReST-EM), §1.3 (DPO), §1.5–1.6 (LoRA, QLoRA), §1.7 (distillation), §2; Pack F §1.4 (Darwin Gödel Machine: archive, parent selection, the test-gaming incident), §2; Pack H §2 (gate calibration).

Tasks 1. `train/dataset.ts`: SFT examples and preference pairs as views over trajectories since the last cut; replay fraction; never reads holdout (I-2); dataset manifest with hashes. 2. `python/qlora_sft.py`, `python/dpo.py`: QLoRA (NF4, double quantization, paged optimizer) on the worker; DPO with the pack's default beta; hyperparameters from Pack D's table, cited; runs on the 3080 within 12 GB. 3. `train/archive.ts`: LoRA archive entries; evaluation on holdout via `Bench`; activation rule (contract §6.6); never merge (I-7). 4. Distillation: senior-solved tasks included as SFT examples for the worker (Pack D §1.7 for the loss where logits are available; otherwise sequence-level). 5. `gate/calibrate.ts`: logistic fit over `gate/decisions.jsonl` with outcomes; asymmetric threshold; switch `gate.mode` to `model` when ≥ 200 labelled decisions exist. 6. Harness evolution: proposals to prompts, retrieval settings and budgets, scored on the dev set, archived with scores (DGM idea); anything touching human-owned paths is a proposal only (I-1). 7. Weekly scheduler: dataset → train → evaluate → archive → report, in the sleep weekly slot.

Acceptance (human runs): a trained LoRA beats the untrained worker on the holdout set at equal cost — or it doesn't, and the archive entry records why.

Session prompt

```
PHASE P5. Read docs/markdown/10-interface-contract.md §6.6, §6.7, §7 (Trainer), §9;
docs/markdown/D-self-training.md
§1.1–1.3, §1.5–1.7, §2; docs/markdown/F-recursive-self-improving.md §1.4, §2; docs/markdown/H-
routing-evaluation.md §2.
Implement tasks P5.1–P5.7 from docs/markdown/20-agent-build-prompt.md. Training reads only
trajectories and dev tasks,
never .tfa/bench/holdout (I-2, test with a canary). The base model is never modified (I-7).
Harness proposals never
touch verifier or benchmark (I-1). Python scripts must be runnable with --dry-run on CPU for
tests. Tests: dataset
view excludes failed candidates and holdout; archive activation rule; calibration on synthetic
decisions.
```

P6 — Experimental track (optional) · open-ended

Read: Pack E (all), Pack F §1.1–1.3, Pack I; contract §7 (`Thinker` plug-in point).

Tasks (each a separate experiment with a benchmark comparison, archived like a LoRA) 1. Titans/HOPE-style neural memory module in PyTorch from the open reimplementations, trained on trajectory data; plugged in as an additional retrieval source. 2. Per-task test-time training (ARC recipe adapted: leave-one-out from the task's own failing tests, LoRA per task) for the hardest queued tasks only. 3. A symbolic planner (PDDL or a small constraint solver) as a tool the Thinker may call for decomposition.

Acceptance: any experiment beats the P5 system on the dev set at equal cost — or is documented as not worth it.

---

4. The human's part between phases

When	What only you can do
Before P0	Create `C3Studium/tfa`; put `docs/` in; run `download-papers.ps1`; install llama.cpp (CUDA build), Node ≥ 20, git; download the worker + draft models; confirm GGUF file names.

When	What only you can do
P0 → P1	Run the P0 acceptance test on a real repo; record attempts and wall-clock. Decide which repos and docs are in scope for the Library.
P1 → P2	Write the 20 dev + 10 holdout tasks as `bench/tasks/*.json` (a failing test or build each). This is the single most important input to the whole project.
P2	Run `tfa bench dev` for the baseline. Download and serve senior A and B; run both on the dev set.
P3	Use the system daily; your corrections are its best data. Approve or reject lesson-test proposals.
P4	Decide RAM upgrade (64 GB lifts the senior class). Review the morning notes; resolve flagged contradictions.
P5	Approve the first LoRA activation thresholds after two weekly runs. Review harness proposals.
Monthly	Rotate holdout: retire to dev, write 10 fresh tasks postdating the last training cut.
Always	`ASK-HUMAN.md` is the agents' channel to you; answer it before starting the next session.

5. Success criteria (from the thesis, restated as numbers to watch)

- First-try verifier pass rate on the dev set: baseline at P2, rising through P3–P5; the holdout curve is the honest one.
- Corrections per task: falling.
- Fast-path share: rising toward the majority of routine work by the end of P4.
- Cost per task (wall-clock, tokens, senior calls): fast path in seconds; slow path budgeted by difficulty.
- Nothing in skills, lessons or training data without a verifier pass; nothing in human-owned paths without a human.